

CSC 8830: Computer Vision - Assignment 6

Optical Flow, Motion Tracking, and Planar Multi-view Geometry

Harshit Jain | Panther ID: 002916196

GitHub repository: github.com/harshitjain25/csc8830-assignment6

1. Objective and Project Setup

This assignment studies motion and multi-view geometry using classical computer vision. Part 1 uses two videos to compute dense optical flow and to track feature points between consecutive frames. Part 2 uses four photographs of a flat book cover to estimate planar transformations and reconstruct the cover boundary in a reference view. The implementation is written in Python using OpenCV and NumPy.

The two input videos are longer than 30 seconds. Optical flow is computed across the full videos, while the report focuses on representative samples and consecutive-frame validation within the first 30 seconds.

Video	FPS	Frames	Duration (s)	Frame size
video1	59.365	1941	32.696	848 x 478
video2	30.112	2236	74.257	848 x 478

2. Part 1: Optical Flow

2.1 Method

Dense Farneback optical flow estimates motion for every pixel between two consecutive grayscale frames [1,2]. The result gives a horizontal component dx and a vertical component dy for each pixel. Motion magnitude is computed from these two components, and the direction is represented by the angle of the motion vector.

$$\text{magnitude} = \sqrt{dx^2 + dy^2}$$

For visualization, HSV color is used. Hue represents motion direction, saturation is fixed at 255, and brightness represents motion magnitude. To make useful motion easier to see, the display scale uses the 95th percentile of non-negligible motion magnitudes. Very small values remain dark. The brightness scale is normalized separately for each frame, so brightness should not be used to compare absolute motion strength across different frames or videos.

2.2 Optical Flow Results

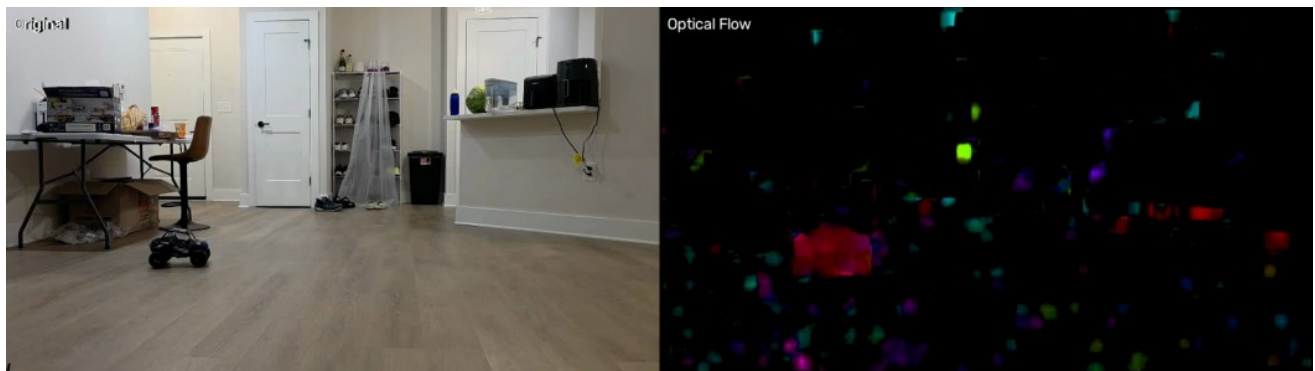


Figure 1. Video 1 at about 10 seconds. The original frame is shown beside the dense optical flow visualization. The small floor-level vehicle is the main moving object.



Figure 2. Video 2 at about 10 seconds. The original frame is shown beside the dense optical flow visualization. Colored regions show apparent image motion.

In the flow images, dark regions have little displayed motion and colored regions indicate stronger motion. Different colors represent different directions. The result is useful for locating moving areas and understanding how motion changes across the frame. Optical flow describes apparent image motion, so it should not be interpreted as exact physical speed in the real world.

3. Motion Tracking from Fundamentals

3.1 Brightness Constancy and the Optical Flow Equation

The basic assumption is brightness constancy. A small image point should keep approximately the same intensity as it moves from one frame to the next [1,3].

$$I(x, y, t) = I(x + \Delta x, y + \Delta y, t + \Delta t)$$

Using a first-order Taylor expansion around the original point gives:

$$I(x + \Delta x, y + \Delta y, t + \Delta t) \approx I + I_x \Delta x + I_y \Delta y + I_t \Delta t$$

After canceling I and dividing by Δt , define $u = \Delta x / \Delta t$ and $v = \Delta y / \Delta t$. This gives the optical flow constraint equation:

$$I_x u + I_y v + I_t = 0$$

One pixel gives one equation but there are two unknown motion values. Lucas-Kanade solves this by assuming that nearby pixels in a small window have almost the same motion. For pixel i , one row of A is $[I_x(i), I_y(i)]$ and the matching entry of b is $-I_t(i)$. For two consecutive frames, $I_t(i)$ is approximated by $I_B(i) - I_A(i)$. The displacement vector is $d = [dx, dy]^T$.

$$A d \approx b$$

$$A^T A d = A^T b$$

$$d = (A^T A)^{-1} A^T b$$

This solution works best when the local window has image variation in both x and y directions. That is why corner-like features are useful for tracking. The OpenCV implementation also uses image pyramids so larger motion can be handled more reliably [1,3].

3.2 Direct Numerical Check on Actual Video Pixels

To connect the theory to the actual videos, the normal equations were also formed directly from saved frame pairs. A 21×21 window, or 441 pixels, was used around a tracked point. Central differences from frame A were used for I_x and I_y . The temporal term was formed from $I_B - I_A$. The values were accumulated into the 2×2 normal system $Gd = c$, where $G = A^T A$ and $c = A^T b$. This calculation was done directly from the saved image intensities, without calling Lucas-Kanade.

$$G = [[\sum I_x^2, \sum I_x I_y], [\sum I_x I_y, \sum I_y^2]]$$

$$c = [-\sum I_x (I_B - I_A), -\sum I_y (I_B - I_A)]^T$$

For video 1 point 1 at (329,141):

$$G = [[183857.50, -12931.25], [-12931.25, 282521.25]]$$

$$c = [-29185.00, -59083.00]^T$$

Solving $Gd = c$ gives $d = (-0.174006, -0.217092)$ pixels, so the first-order prediction is (328.825994, 140.782908).

For video 2 point 1 at (239,145):

$$G = [[421217.75, -51913.25], [-51913.25, 473184.25]]$$

$$c = [257197.00, -53723.00]^T$$

Solving $Gd = c$ gives $d = (0.604788, -0.047183)$ pixels, so the first-order prediction is (239.604788, 144.952817).

Video	Start point	Direct solve d=(dx,dy)	Direct prediction	LK prediction
video1	(329, 141)	(-0.1740, -0.2171)	(328.8260, 140.7829)	(328.8341, 140.8060)
video2	(239, 145)	(0.6048, -0.0472)	(239.6048, 144.9528)	(239.5198, 144.9972)

The direct first-order result and the pyramidal Lucas-Kanade result are close but not identical. This is expected because the OpenCV tracker uses iterative refinement, interpolation, and multiple pyramid scales.

4. Bilinear Interpolation

During tracking, a predicted point can fall between integer pixel locations. Bilinear interpolation estimates the intensity at that subpixel position using the four nearest pixels [5]. Let $x_0 = \text{floor}(x)$, $y_0 = \text{floor}(y)$, $a = x - x_0$, and $b = y - y_0$.

$$I(x,y) = (1-a)(1-b)I_{00} + a(1-b)I_{10} + (1-a)bI_{01} + abI_{11}$$

Example: if the four intensities are 100, 120, 140, and 160, and the sample location is (10.25, 20.5), then $a = 0.25$ and $b = 0.5$. The bilinear estimate is 125. This allows the tracker to compare image values at subpixel coordinates instead of forcing every location to the nearest integer pixel.

5. Consecutive-Frame Tracking Validation

5.1 Independent Comparison Method

The Lucas-Kanade result was checked with a separate image-patch search. For each selected feature, a 15 x 15 grayscale patch from frame A was searched within plus or minus 45 pixels in frame B using normalized correlation with `cv2.TM_CCOEFF_NORMED` [6]. The comparison did not use the Lucas-Kanade destination. A match was treated as high confidence only when the source patch standard deviation was at least 5, the correlation score was at least 0.95, and the best match was at least 0.05 better than competing matches more than 3 pixels away. These comparison locations are automated integer-pixel estimates, not manually measured ground truth. They provide a useful independent check, but they also have their own limitations.

5.2 Video 1

For video 1, the script searched sampled frame pairs in the first 30 seconds and selected frames 1770 and 1771 because they had the largest mean grayscale difference among the sampled pairs. Even this pair had weak overall change, so the selected feature points were mostly on the static background.

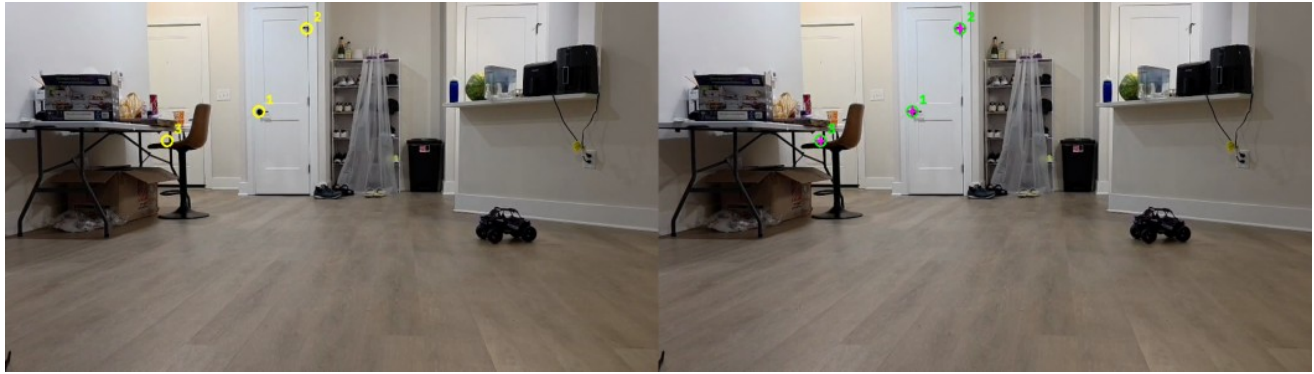


Figure 3. Video 1 tracking validation. Yellow points are initial locations, green points are independently matched pixel locations, and magenta points are Lucas-Kanade predictions.

ID	Frame A	LK prediction	Independent match	NCC	Error (px)
1	(329,141)	(328.834,140.806)	(329,141)	0.9876	0.255
2	(391,34)	(390.945,33.893)	(391,34)	0.9939	0.120
3	(210,179)	(209.981,178.839)	(210,179)	0.9968	0.162

For point 1, $dx = -0.1659$ pixels and $dy = -0.1940$ pixels. The displacement magnitude is 0.2553 pixels. The independent patch match stays at the same integer pixel, so the disagreement is also 0.2553 pixels. The high-confidence matches had a mean disagreement of 0.160 pixels. This mainly validates small background motion rather than tracking the moving vehicle itself.

5.3 Video 2

For video 2, the selected pair was frames 450 and 451 at about 14.94 seconds. The tracker again detected 100 points. Most high-confidence points showed small disagreement between Lucas-Kanade and an independent patch search.

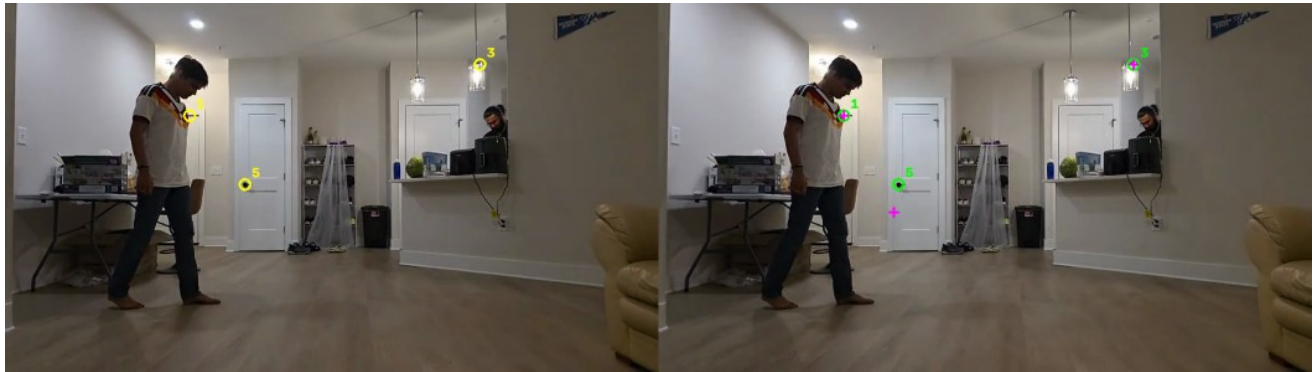


Figure 4. Video 2 tracking validation. Yellow points are initial locations, green points are independently matched pixel locations, and magenta points are Lucas-Kanade predictions.

ID	Frame A	LK prediction	Independent match	NCC	Error (px)
1	(239,145)	(239.520,144.997)	(240,145)	0.9685	0.480
3	(616,79)	(615.998,79.000)	(616,79)	1.0000	0.002
5	(311,234)	(304.570,270.306)	(311,234)	0.9963	36.872

For point 1, the Lucas-Kanade displacement magnitude is about 0.520 pixels and the independent pixel match differs by about 0.480 pixels. Across the high-confidence matches, the mean discrepancy is 0.560 pixels and the median is 0.091 pixels.

5.4 Tracking Limitation Found in Video 2

Point 5 in video 2 is a useful failure example. It starts on the door handle at (311,234), but Lucas-Kanade predicts a large movement to about (304.570,270.306). The independent patch search finds the same door-handle position at (311,234) with high correlation. This suggests that the large Lucas-Kanade movement is a mismatch rather than real object motion.



Figure 5. Close-up of the door-handle feature. Yellow marks the starting feature, green marks the independent match, and magenta marks the Lucas-Kanade prediction.

This result is important because a successful tracker status does not always mean the motion estimate is correct. Repeated texture, lighting changes, occlusion, and weak local image structure can still cause errors. The independent patch search is also not perfect, but it provides a useful second check.

6. Part 2: Planar Structure from Motion

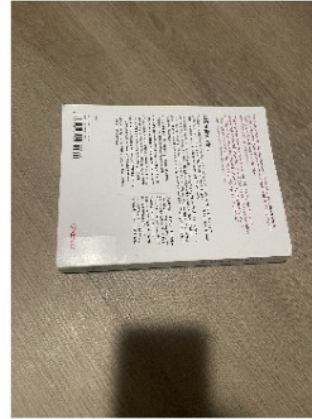
6.1 Object and Camera Setup

A flat book back cover was photographed from four different viewpoints. The back cover was treated as a plane. The book thickness, spine, and protruding pages were not used for the planar calculation. Four cover corners were used to fit each homography, and three extra printed points were used only for independent validation.

view1



view2



view3



view4

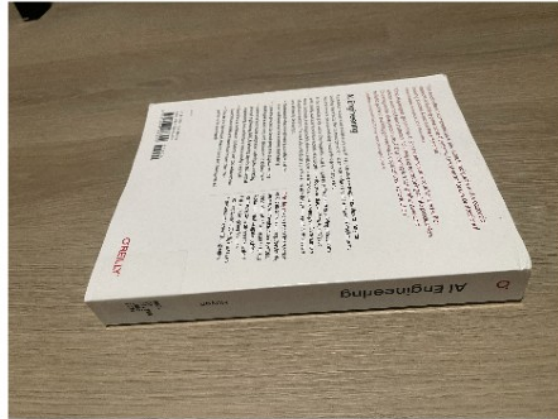


Figure 6. Four photographs of the same planar book cover from different viewpoints.

View	Relative viewpoint	Decoded image size
view1	Oblique overhead	3024 x 4032
view2	Closer to overhead	3024 x 4032
view3	Rotated oblique	3024 x 4032
view4	Lower grazing angle	4032 x 3024

The image metadata shows an Apple iPhone 12 Pro Max using the 5.1 mm f/1.6 back camera, with a 25 mm full-frame equivalent focal length. The recorded exposure was 1/60 s and ISO 200. Exact camera-to-object distances and viewing angles were not measured, so this experiment reports a planar projective reconstruction rather than a metric 3D reconstruction.

6.2 Homography Model and Derivation

For a planar surface, corresponding points in two views are related by a 3 x 3 homography matrix H [4]. View 1 is used as the reference coordinate system.

$$s [x', y', 1]^T = H [x, y, 1]^T$$

Let H contain eight independent parameters after fixing the bottom-right entry to 1. For a source point (x, y) , multiply by H and divide by the third homogeneous coordinate to obtain the projected reference point:

$$x' = (h_{11}x + h_{12}y + h_{13}) / (h_{31}x + h_{32}y + 1)$$

$$y' = (h_{21}x + h_{22}y + h_{23}) / (h_{31}x + h_{32}y + 1)$$

Each point correspondence gives two linear equations. Four non-collinear corner correspondences provide eight equations, which are enough to estimate the eight independent parameters. If $h = [h_{11}, h_{12}, h_{13}, h_{21}, h_{22}, h_{23}, h_{31}, h_{32}]^T$, one correspondence (x, y) to (x_prime, y_prime) contributes these two rows:

$$[x, y, 1, 0, 0, 0, -x x_prime, -y x_prime] h = x_prime$$

$$[0, 0, 0, x, y, 1, -x y_prime, -y y_prime] h = y_prime$$

For example, view 2 corner 1 has source point (2460.098389, 1017.304626) and reference point (2788.524658, 1736.861572). The x-equation therefore contains $-x x_prime = -6860045.018$ and $-y x_prime = -2836779.036$. The remaining seven rows are formed in the same way. The implementation uses `cv2.findHomography` with the four manually selected corner pairs and `cv2.perspectiveTransform` to project points into view 1.

6.3 Selected Corners and Estimated Homographies

View	C1	C2	C3	C4
view1	(2788.5,1736.9)	(1127.8,2735.6)	(142.5,1085.5)	(1462.4,514.9)
view2	(2460.1,1017.3)	(2726.6,2444.0)	(452.4,2580.5)	(501.9,1004.9)
view3	(2131.7,3455.1)	(545.3,3262.8)	(650.7,1253.0)	(1933.4,657.5)
view4	(3157.7,525.7)	(3776.2,1916.8)	(553.4,2168.0)	(1027.8,739.7)

The fitted homography matrices that map each source view into view 1 are:

$$H_{2 \rightarrow 1} = [[0.208923163175, -0.785332590124, 2075.596690917631], [0.316947972581, 0.393128641170, -64.332461071048], [-0.000158098893, 0.000030558114, 1.000000000000]]$$

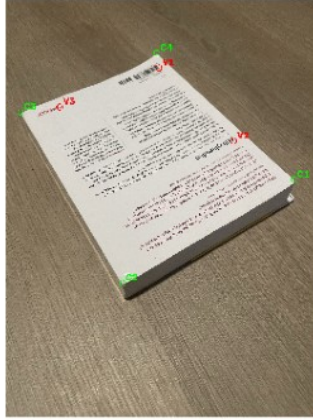
$$H_{3 \rightarrow 1} = [[2.239382, 0.599762, -2036.591289], [0.054619, 0.728695, 361.357372], [0.000458, -0.000073, 1.000000]]$$

$$H_{4 \rightarrow 1} = [[0.244179, -0.870671, 1958.324247], [0.493593, 0.875924, -604.184647], [-0.000097, 0.000230, 1.000000]]$$

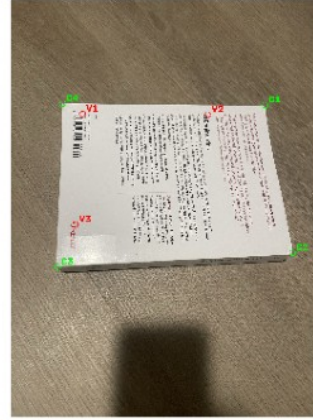
6.4 Independent Validation

The four fitting corners produce almost zero fitting error because they are the same points used to estimate H . To test the transformation more meaningfully, three additional printed points on the book cover were selected in every view. These validation points were not used when fitting the homography.

view1



view2



view3



view4

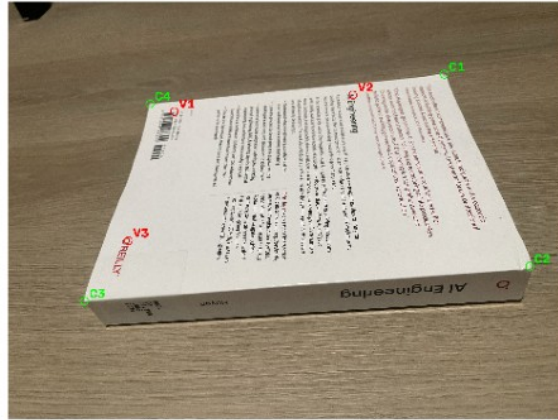


Figure 7. Green labels C1-C4 show fitting corners. Red labels V1-V3 show independent validation points.

View	Validation 1	Validation 2	Validation 3	Mean error (px)
view2	25.240	34.921	15.254	25.139
view3	34.415	24.800	18.858	26.024
view4	21.976	46.321	21.111	29.803

The overall independent mean reprojection error is 26.989 pixels. A worked example is view 2 validation point 1. Its source coordinate is $p = [694.032776, 1104.147705, 1]^T$. Using $H_2 \rightarrow 1$ gives the homogeneous coordinates:

$$X = 0.208923163175(694.032776) - 0.785332590124(1104.147705) + 2075.596690917631 = 1353.473037$$

$$Y = 0.316947972581(694.032776) + 0.393128641170(1104.147705) - 64.332461071048 = 589.711907$$

$$W = -0.000158098893(694.032776) + 0.000030558114(1104.147705) + 1.000000000000 = 0.924014858$$

Divide by W to return to image coordinates. This gives the predicted point (1464.774, 638.206). The manually selected reference point is (1481.016, 657.526).

$$e = \sqrt{((1464.774 - 1481.016)^2 + (638.206 - 657.526)^2)} = 25.240 \text{ pixels}$$

The individual validation errors range from 15.254 to 46.321 pixels. Possible contributors to these errors include manual clicking uncertainty, image resampling, lens distortion, and small deviations from ideal planarity. Their individual effects were not measured. The errors should be treated as image-space agreement, not as metric 3D accuracy.

6.5 Reconstructed Boundary

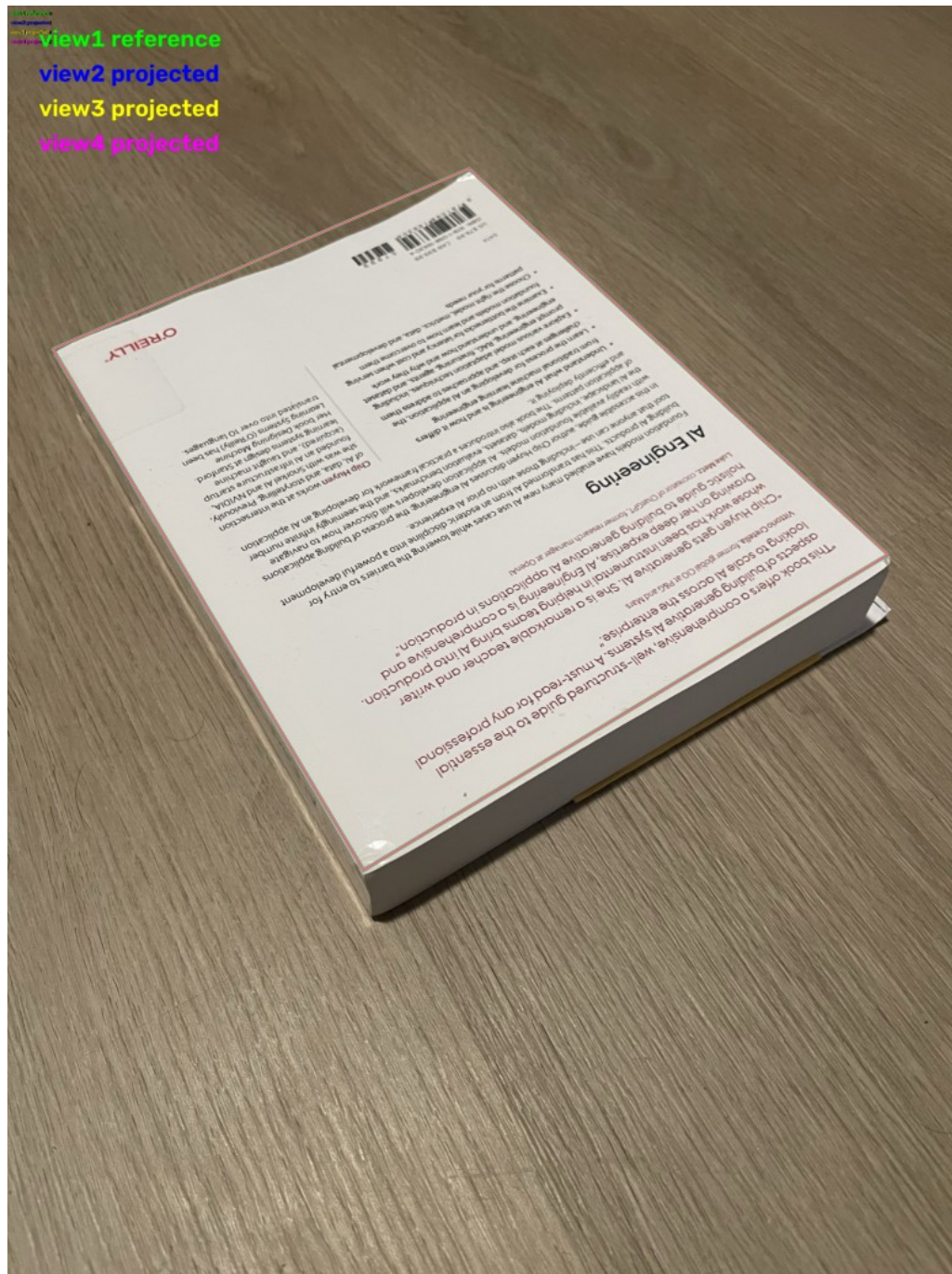


Figure 8. Reference boundary in view 1 with the projected boundaries from views 2, 3, and 4. The projected outlines nearly overlap because the four cover corners were used to fit each homography.

The projected boundaries from the other three views align closely with the reference book boundary. This shows that a single planar homography can map the cover between these viewpoints. The independent interior points are more useful for measuring accuracy because the boundary corners were already used during fitting.

7. Discussion

The optical flow visualization successfully shows where image motion occurs and the direction of that motion. Lucas-Kanade tracking provides subpixel displacement estimates, but the results also show that tracking can fail even when OpenCV reports a successful track. Using a second matching method helped identify one large false displacement in video 2.

The planar multi-view experiment shows that the same flat object can be mapped across four camera viewpoints with homographies. The reconstructed boundary is visually consistent, and the held-out validation points give an overall mean error of 26.989 pixels. Since the camera was not geometrically calibrated and exact camera poses were not measured, the result should be described as planar projective geometry rather than full metric structure from motion.

8. Conclusion

This assignment demonstrated three connected computer vision ideas. Dense optical flow was used to visualize motion in two videos. Lucas-Kanade tracking was derived from brightness constancy and tested on consecutive frames, including a clear example of a tracking mismatch. Finally, four views of a flat book cover were related using planar homographies, and independent validation points were used to measure reprojection error. The experiments show both what these classical methods can do and where their assumptions can limit accuracy.

9. References

- [1] OpenCV. Optical Flow. https://docs.opencv.org/4.x/d4/dee/tutorial_optical_flow.html
 - [2] G. Farneback. Two-Frame Motion Estimation Based on Polynomial Expansion. SCIA 2003, pp. 363-370. DOI: 10.1007/3-540-45103-X_50.
 - [3] B. D. Lucas and T. Kanade. An Iterative Image Registration Technique with an Application to Stereo Vision. IJCAI 1981, pp. 674-679. <https://publications.ri.cmu.edu/an-iterative-image-registration-technique-with-an-application-to-stereo-vision-ijcai>
 - [4] OpenCV. Basic concepts of the homography explained with code. https://docs.opencv.org/4.x/d9/dab/tutorial_homography.html
 - [5] OpenCV. Geometric Image Transformations, INTER_LINEAR bilinear interpolation. https://docs.opencv.org/4.x/da/d54/group_imgproc_transform.html
 - [6] OpenCV. Template Matching. https://docs.opencv.org/4.x/d4/dc6/tutorial_py_template_matching.html
- Course supplementary resource: First Principles of Computer Vision playlist on Optical Flow, Structure from Motion, and Object Tracking. <https://www.youtube.com/playlist?list=PL2zRqk16wsdoYzrWStffqBAoUY8XdvatV>